UNIVERSITY OF IBADAN

DEPARTMENT OF COMPUTER SCIENCE

CSC 790: DEEP LEARNING

COURSE PROJECT REPORT

DEEP NEURAL NETWORKS FOR SEMANTIC SENTIMENT ANALYSIS IN CODE-MIXED ENGLISH-YORÙBÁ SOCIAL MEDIA CONVERSATIONS

BY

SAHEED OLAYEMI OLAYINKA

Matriculation Number: 214905

SAMSON AKINYELE FAGBENJO

Matriculation Number: 261743

Submitted to: Dr. Oludele Adeleke (Course Lecturer)

November 2026

ABSTRACT

Nigerians who write on social media in Yorùbá often switch into English within the same post. This project asks how well deep neural networks read the sentiment of such posts, and whether the switching itself makes the task harder. We used the Yorùbá portion of the AfriSenti-SemEval 2023 Twitter benchmark (8,522 training, 2,090 development and 4,515 test tweets, each labelled positive, neutral or negative). Because the released data say nothing about which tweets are code-mixed, the two authors, both fluent in Yorùbá, annotated every test tweet that a lexicon-based Code-Mixing Index flagged as a candidate (1,978 tweets) and a random audit of 400 of the rest. Agreement between the annotators was substantial (Cohen's kappa 0.71). We then trained six models under one protocol with five random seeds each: an LSTM and a bidirectional LSTM with attention trained from scratch, and four fine-tuned transformers (mBERT, XLM-RoBERTa base, AfriBERTa-large and AfroXLMR-large).

The Africa-centric transformers performed best, with weighted F1 of 73.6 (AfroXLMR-large) and 72.5 (AfriBERTa-large); the two are statistically tied. A plain LSTM (67.8) beat both general multilingual transformers, mBERT (63.6) and XLM-RoBERTa base (61.7). Genuine code-mixing turned out to be rarer than the heuristic suggests: 6.0% of test tweets are mixed, and another 6.9% pair a Yorùbá line with its English translation. No model scored significantly worse on mixed tweets, although the recurrent models and mBERT dropped by four to five points while the Africa-centric models dropped by about one. Tweets built around proverbs, idioms and praise poetry were the hardest for every model; the recurrent models lost about 13 to 15 points on them and AfroXLMR-large about 6. These results indicate that contextual, Africa-centric pretrained models are the better choice for Yorùbá sentiment analysis in applications such as brand monitoring and policy sentiment tracking, and that cultural expressions, rather than switching between languages, are the main remaining source of error.

Keywords: sentiment analysis; code-mixing; Yorùbá; LSTM; attention; transformers; AfriBERTa; XLM-RoBERTa; low-resource languages

TABLE OF CONTENTS

ABSTRACT.....ii

TABLE OF CONTENTS.....iii

CHAPTER ONE.....1

INTRODUCTION.....1

1.1 Background to the Study.....1

1.2 Statement of the Problem.....2

1.3 Aim and Objectives.....2

1.4 Research Questions.....2

1.5 Significance of the Study.....2

1.6 Scope and Limitations.....3

1.7 Definition of Terms.....3

CHAPTER TWO.....4

LITERATURE REVIEW.....4

2.1 Sentiment Analysis.....4

2.2 Code-Mixing in Social Media Text.....4

2.3 Deep Learning Architectures for Text Classification.....4

2.3.1 Long Short-Term Memory Networks.....4

2.3.2 Bidirectional LSTM with Attention.....5

2.3.3 Transformers and Contextual Embeddings.....5

2.3.4 Multilingual and Africa-Centric Pretrained Models.....5

2.4 Empirical Review.....5

2.5 Research Gap.....6

CHAPTER THREE.....7

METHODOLOGY.....7

3.1 Research Design.....7

3.2 Dataset..... 7
3.3 Pre-processing and Data Audit..... 7
3.4 Code-Mixing Annotation..... 8
3.4.1 Code-Mixing Index..... 8
3.4.2 Annotation Guidelines..... 8
3.4.3 Sampling and Agreement..... 8
3.5 Model Architectures..... 9
3.5.1 LSTM..... 9
3.5.2 Bidirectional LSTM with Attention..... 9
3.5.3 Fine-Tuned Transformers..... 9
3.6 Training Procedure..... 10
3.7 Evaluation Metrics..... 10
3.8 Statistical Analysis..... 11
3.9 Tools..... 11
3.10 Ethical Considerations..... 11
CHAPTER FOUR..... 12
RESULTS AND DISCUSSION..... 12
4.1 Overall Model Performance..... 12
4.2 Performance by Sentiment Class..... 13
4.3 Statistical Significance of the Differences..... 14
4.4 Prevalence of Code-Mixing..... 14
4.5 Effect of Code-Mixing on Sentiment Accuracy (RQ1)..... 15
4.6 Local Expressions and Idioms (RQ3)..... 16
4.7 Error Analysis of the Best Model..... 17
4.8 What the Attention Model Looks At..... 18
4.9 Discussion..... 19

CHAPTER FIVE....20
SUMMARY, CONCLUSION AND RECOMMENDATIONS....20
5.1 Summary....20
5.2 Conclusion....20
5.3 Recommendations....20
5.4 Contributions to Knowledge....21
5.5 Limitations....21
5.6 Suggestions for Further Work....21
REFERENCES....22
APPENDIX A: TRAINING ENVIRONMENT AND RUN TIMES....24
APPENDIX B: RESULTS OF EVERY RUN....25
APPENDIX C: CODE-MIXING ANNOTATION GUIDELINES....26
APPENDIX D: CODE PACKAGE AND HOW TO RUN IT....27

CHAPTER ONE

INTRODUCTION

1.1 Background to the Study

Social media has become one of the main places where Nigerians talk about prices, politicians, brands and everyday life. Much of that talk is not in a single language. A Yorùbá speaker in Ibadan or Abeokuta may open a post in Yorùbá, finish it in English, drop an English phrase in for emphasis, or write a proverb and then translate it for friends who do not speak the language. Linguists call this code-mixing or code-switching. It is ordinary behaviour for bilingual speakers, but it is awkward for language technology, because most sentiment analysis systems were built for one language at a time and most were built for English.

Sentiment analysis is the task of deciding whether a piece of text expresses a positive, negative or neutral attitude (Liu, 2012). For Yorùbá the task is harder than for English for several reasons. Yorùbá is a tonal language whose written form uses diacritics that are often dropped on social media, so the same spelling can stand for different words. It has far less digital text than English, which limits what models can learn. And Yorùbá speech is rich in proverbs (òwe), praise poetry (oríkì) and idioms whose meaning is not the sum of their words.

Deep learning has changed what is possible in this setting. Recurrent neural networks such as the Long Short-Term Memory network (LSTM; Hochreiter and Schmidhuber, 1997) read text word by word and can learn sentiment cues directly from labelled examples. Attention mechanisms (Bahdanau et al., 2015) let such networks focus on the words that matter most. Transformer models pretrained on large multilingual corpora, such as multilingual BERT (Devlin et al., 2019) and XLM-RoBERTa (Conneau et al., 2020), produce contextual embeddings, in which the representation of a word depends on the words around it. Models pretrained specifically on African languages, such as AfriBERTa (Ogueji et al., 2021) and AfroXLMR (Alabi et al., 2022), aim to bring those benefits to languages that the general models cover poorly.

The AfriSenti benchmark (Muhammad et al., 2023b) and its predecessor NaijaSenti (Muhammad et al., 2022) made it possible to train and compare such models on Yorùbá tweets. Their authors note that the data contain code-mixed tweets, but the released files carry only a sentiment label, so no one can tell from the data which tweets are mixed or whether models handle them worse. This project fills that gap and uses it to answer the three research questions set for this topic.

1.2 Statement of the Problem

Organisations that want to know how Nigerians feel about a product, a policy or a candidate need sentiment tools that work on the language people actually use online. For Yorùbá, that language is frequently mixed with English. It is not known how much this mixing affects deep learning sentiment models, which architecture copes best with it, or whether contextual embeddings help with the proverbs and idioms that carry so much meaning in Yorùbá. Without that knowledge, a practitioner choosing a model for, say, tracking public reaction to a fuel price increase has no evidence on which to base the choice. The public Yorùbá sentiment data cannot answer these questions as released, because they do not mark which tweets are code-mixed.

1.3 Aim and Objectives

The aim of this project is to evaluate deep neural network architectures for sentiment analysis of English-Yorùbá social media text and to measure how code-mixing and local expressions affect them. The specific objectives are to:

1. build a human-verified code-mixing annotation of the AfriSenti Yorùbá test set and measure how common mixing is;
2. implement and train an LSTM and a bidirectional LSTM with attention, and fine-tune AfriBERTa and XLM-RoBERTa (with mBERT and AfroXLMR as additional reference models), under one fair protocol;
3. compare the models on the full test set and on code-mixed and non-mixed tweets, with statistical tests;
4. assess whether contextual models handle proverbs, idioms and other local expressions better than non-contextual ones; and
5. analyse the errors of the best model and interpret what the attention model focuses on.

1.4 Research Questions

- RQ1. How does code-mixing affect semantic interpretation by NLP models?
- RQ2. Which deep learning architecture performs best on mixed-language sentiment analysis?
- RQ3. Can contextual embeddings improve understanding of local expressions and idioms?

1.5 Significance of the Study

The findings matter to anyone who builds or buys sentiment analysis for Nigerian audiences. In brand monitoring, a telecoms or banking company tracking complaints about network outages or

failed transfers needs to know whether its tool will misread posts that switch between Yorùbá and English. In public opinion analysis, a ministry or newspaper measuring reaction to a policy, such as a rise in the pump price of petrol (one test tweet reads simply "owó epo léwó owó ọkọ̀ léwó", fuel is expensive, transport is expensive), needs a model that reads Yorùbá correctly. In election and policy sentiment tracking, civil society groups monitoring the tone of political conversation in the South-West need evidence about which models to trust. The project also contributes a reusable resource: the code-mixing labels for the test set, which other researchers can use to evaluate their own models.

1.6 Scope and Limitations

The study covers Yorùbá and English on Twitter, using the AfriSenti-SemEval 2023 Yorùbá data, and three-way sentiment (positive, neutral, negative). It evaluates six architectures with one training recipe each. Code-mixing is annotated at the level of the whole tweet, not word by word, and only on the test set. The annotators are the two authors. The local-expression analysis relies on tweets that explicitly mention proverbs, idioms, sayings or praise poetry, which is a practical but partial way to find such expressions.

1.7 Definition of Terms

- Code-mixing: the use of two languages within one utterance or post, here Yorùbá and English.
- Translation pair: a tweet in which a Yorùbá line is followed by its English translation; we treat it as a separate case from natural mixing.
- Code-Mixing Index (CMI): a score from 0 to 100 that measures how much of a text comes from languages other than its dominant one (Das and Gambäck, 2014).
- Contextual embedding: a vector representation of a word that changes with the surrounding words, as produced by transformer models.
- Fine-tuning: further training of a pretrained model on a labelled task, here sentiment classification.
- Weighted F1: the F1 score of each class averaged with weights proportional to the number of examples in that class; the standard metric of the AfriSenti shared task.
- Cohen's kappa: a measure of agreement between two annotators that corrects for agreement expected by chance (Cohen, 1960).

CHAPTER TWO

LITERATURE REVIEW

2.1 Sentiment Analysis

Sentiment analysis grew out of work on opinion mining in product reviews and has become a standard task in natural language processing (Pang and Lee, 2008; Liu, 2012). Early systems used word lists and hand-built rules, then machine learning over bag-of-words features. These approaches struggle with negation, sarcasm and words whose polarity depends on context. Deep learning replaced hand-built features with representations learnt from data, which made the task more accurate but also more dependent on the amount and kind of training text available.

2.2 Code-Mixing in Social Media Text

Code-mixing is common wherever bilingual communities write informally, and it raises specific problems for NLP: words from two vocabularies, spelling that follows neither language's standard, and meaning that can shift at the switch point. A widely used measure of how mixed a text is, the Code-Mixing Index, compares the number of words in the dominant language with the number of language-tagged words in the text (Das and Gambäck, 2014). Most code-mixed sentiment resources cover Hindi-English and Spanish-English; the SemEval-2020 Task 9 shared task released labelled tweets for both pairs and drew many neural systems (Patwa et al., 2020). A recent survey shows how the field has grown and how unevenly it covers the world's language pairs, with African pairs among the least studied (Winata et al., 2023). The wider gap in resources for African languages has also been documented (Joshi et al., 2020).

2.3 Deep Learning Architectures for Text Classification

2.3.1 Long Short-Term Memory Networks

Recurrent neural networks read a sequence one token at a time and carry a hidden state forward. Plain recurrent networks lose information over long sequences because gradients shrink as they pass back through time. The LSTM addresses this with a memory cell and three gates, input, forget and output, that control what is written to, kept in and read from the cell (Hochreiter and Schmidhuber, 1997). For sentiment classification, the final hidden state is passed to a classifier. An LSTM learns its word embeddings from the training data alone unless pretrained vectors are supplied, so it knows nothing about words it has not seen.

2.3.2 Bidirectional LSTM with Attention

A bidirectional LSTM runs one LSTM forwards and another backwards over the text and joins their states, so each position sees both left and right context. An attention layer then scores every position and builds the tweet representation as a weighted sum of the states (Bahdanau et al., 2015). Attention usually helps when the decisive words are far from the end of the text, and the weights offer a rough view of what the model relied on.

2.3.3 Transformers and Contextual Embeddings

The transformer replaces recurrence with self-attention, in which every token attends to every other token in parallel (Vaswani et al., 2017). Pretraining a transformer on large amounts of unlabelled text with a masked language modelling objective, as in BERT (Devlin et al., 2019), produces contextual embeddings: the vector for a word depends on its sentence. Fine-tuning such a model on a small labelled dataset usually beats training a network from scratch, because the model arrives already knowing a great deal about the language.

2.3.4 Multilingual and Africa-Centric Pretrained Models

Multilingual BERT and XLM-RoBERTa were pretrained on text in about one hundred languages (Devlin et al., 2019; Conneau et al., 2020). African languages make up a small share of that text, so their coverage is thin. AfriBERTa took the opposite approach and pretrained a model from scratch on a small corpus of African languages, showing that less but better-targeted data can compete (Ogueji et al., 2021). AfroXLMR instead starts from XLM-RoBERTa and continues its pretraining on African languages, a method its authors call multilingual adaptive fine-tuning (Alabi et al., 2022). Both have been strong baselines on African-language tasks.

2.4 Empirical Review

Table 2.1 summarises the studies closest to this project. NaijaSenti introduced Twitter sentiment corpora for Hausa, Igbo, Nigerian Pidgin and Yorùbá (Muhammad et al., 2022). AfriSenti extended the effort to 14 African languages (Muhammad et al., 2023b) and became the data for SemEval-2023 Task 12 (Muhammad et al., 2023c). In the AfriSenti experiments on Yorùbá, the Africa-centric models were the strongest single-model baselines. Studies on fine-tuning stability show that transformer results can vary widely across random seeds on small datasets, which argues for reporting several runs (Dodge et al., 2020).

Table 2.1 Summary of related studies

Study	Data / language	Method	Relevance to this project
Muhammad et al. (2022), NaijaSenti	Hausa, Igbo, Pidgin, Yorùbá tweets	Baseline classifiers including pretrained transformers	Source of the Yorùbá tweets; notes code-mixed tweets but gives no per-tweet label
Muhammad et al. (2023b), AfriSenti	14 African languages	Multilingual and Africa-centric transformers	Benchmark split used here; Africa-centric models strongest on Yorùbá
Muhammad et al. (2023c), SemEval-2023 Task 12	AfriSenti	Shared task with many systems	Names code-switching as a challenge; no code-mixing analysis
Patwa et al. (2020), SemEval-2020 Task 9	Hindi-English, Spanish-English tweets	Shared task, mostly neural systems	Main earlier code-mixed sentiment benchmark; no African pairs
Ogueji et al. (2021), AfriBERTa	11 African languages	Transformer pretrained from scratch on small data	One of the lecturer's required models
Alabi et al. (2022), AfroXLMR	African languages	XLM-R adapted to African languages	Strongest published Yorùbá baseline; used as reference

2.5 Research Gap

Previous work has built Yorùbá sentiment data and benchmarked models on it, and separate work has studied code-mixed sentiment for other language pairs. No study we found measures how often English-Yorùbá mixing occurs in the AfriSenti data or whether it lowers model accuracy, because the data carry no mixing labels. The lecturer's three questions, about the effect of mixing, the best architecture and the value of contextual embeddings for local expressions, therefore remain open for Yorùbá. This project answers them with a new annotation, a controlled comparison of recurrent and transformer models, and statistical tests.

CHAPTER THREE

METHODOLOGY

3.1 Research Design

The study is a quantitative, experimental comparison. The work ran in four stages. First, we downloaded the benchmark and audited it for duplicates. Second, we computed a Code-Mixing Index for every tweet and used it to select tweets for human annotation. Third, we trained six models with five random seeds each on the same data. Fourth, we evaluated every model on the full test set and on the subsets defined by the annotation, and tested the differences statistically. All work was done in Python notebooks on free cloud GPUs (Google Colab and Kaggle, NVIDIA Tesla T4).

3.2 Dataset

We used the Yorùbá data of AfriSenti-SemEval 2023, released under a Creative Commons Attribution 4.0 licence (Muhammad et al., 2023b). We confirmed that these files are identical to the Yorùbá files in the NaijaSenti repository. Each tweet carries one of three labels. User names are already replaced with @user. Table 3.1 gives the size and label balance of each split.

Table 3.1 AfriSenti Yorùbá splits and label distribution

Split	Tweets	Positive	Neutral	Negative
Training	8,522	41.6%	36.5%	22.0%
Development	2,090	42.3%	36.5%	21.2%
Test	4,515	42.5%	35.8%	21.7%

3.3 Pre-processing and Data Audit

The only cleaning applied to the text was the removal of escaped double quotes left in the files. Tone marks, emojis and capitalisation were kept for the transformers; the recurrent models lower-case the text, replace links with a single token and split it into words and punctuation. Tweets average 22 words.

Before training we checked whether any tweets appear in more than one split, because a model scored on tweets it has already seen will look better than it is. After lower-casing and collapsing spaces, 270 development tweets (12.9%) turned out to be copies of training tweets, and 12 test tweets did as well. Since the development set decides when training stops, we used a deduplicated development set of 1,820 tweets for that purpose. Removing the 12 duplicates from the test set

changed every model's score by less than 0.1 points, so we report the full test set, which keeps our numbers comparable with published work.

3.4 Code-Mixing Annotation

3.4.1 Code-Mixing Index

Each word was tagged as Yorùbá, English or language-neutral (user names, links, numbers, hashtags) using an English word list, a list of Yorùbá stop words and the presence of Yorùbá diacritics. The index follows Das and Gambäck (2014):

$$\mathrm{CMI} = 100 \times (1 - \max(w_i) / (n - u)) \qquad (3.1)$$

where max(w_i) is the number of words in the most frequent language, n the number of tokens and u the number of language-neutral tokens. A tweet entirely in one language scores 0.

3.4.2 Annotation Guidelines

Each tweet was labelled 1 if the writer naturally mixes Yorùbá and English, and 0 otherwise. A tweet counts as 0 when it is in one language only, when English appears only in hashtags, links or names, when the only English is one ordinary borrowed word such as "phone" or "okay", or when a Yorùbá line is simply followed by its English translation; this last case was marked separately as a translation pair. For mixed tweets, the annotator also recorded the dominant language: mostly Yorùbá, mostly English, or balanced (neither language above about two thirds of the words). Annotators judged language only, never sentiment, and did not see the sentiment label or the index. The full guidelines are in Appendix C.

3.4.3 Sampling and Agreement

All 1,978 test tweets with an index above zero (group S1) were annotated. Of the 2,537 tweets with an index of zero (group S0), a random sample of 400 was annotated to check how many mixed tweets the index misses. The remaining tweets were split at random between the two annotators, and 293 tweets were labelled by both to measure agreement with Cohen's kappa:

$$\kappa = (p_o - p_e) / (1 - p_e) \qquad (3.2)$$

where p_o is the observed proportion of agreement and p_e the agreement expected by chance from each annotator's label rates. Agreement was computed before the annotators discussed anything; disagreements were then settled together.

3.5 Model Architectures

3.5.1 LSTM

The LSTM maps each word to a 200-dimensional embedding learnt from scratch, reads the tweet with 128 hidden units, and passes the final hidden state through dropout (0.3) to a softmax layer over the three classes. At each step t the LSTM computes:

$$i_t = \sigma(W_i x_t + U_i h_{(t-1)} + b_i)$$
$$f_t = \sigma(W_f x_t + U_f h_{(t-1)} + b_f)$$
$$o_t = \sigma(W_o x_t + U_o h_{(t-1)} + b_o)$$
$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{(t-1)} + b_c)$$
$$c_t = f_t \odot c_{(t-1)} + i_t \odot \tilde{c}_t$$
$$h_t = o_t \odot \tanh(c_t) \qquad (3.3)$$

where σ is the logistic sigmoid, ⊙ element-wise multiplication, x_t the input embedding, and i, f and o the input, forget and output gates. The vocabulary has 9,890 entries: every word that appears at least twice in the training data, plus padding and unknown-word tokens.

3.5.2 Bidirectional LSTM with Attention

The second model runs the LSTM in both directions and joins the two hidden states, $h_t = [h\rightarrow_t ; h\leftarrow_t]$. An additive attention layer scores each position and forms the tweet vector as a weighted sum:

$$e_t = v^T \tanh(W h_t)$$
$$\alpha_t = \exp(e_t) / \Sigma_k \exp(e_k)$$
$$s = \Sigma_t \alpha_t h_t \qquad (3.4)$$

Padding positions are masked out before the softmax. The vector s goes through dropout to the classifier. We saved the three most-attended words of every test tweet for the interpretation in Chapter Four.

3.5.3 Fine-Tuned Transformers

Each transformer encodes the tweet, and the vector of its first token passes to a small classification head; all weights are updated during fine-tuning. Self-attention, the core operation, is:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V \qquad (3.5)$$

where Q, K and V are the query, key and value projections of the token vectors and d_k their dimension (Vaswani et al., 2017). Table 3.2 lists the four pretrained models. AfriBERTa and XLM-

RoBERTa are the lecturer's required models; mBERT and AfroXLMR were added as reference points, a general model with Yorùbá in its pretraining and the strongest published AfriSenti baseline for Yorùbá.

Table 3.2 Models compared

Model	Type	Pretrained checkpoint	Learning rate
LSTM	Recurrent, from scratch	none	1e-3 (Adam)
BiLSTM + attention	Recurrent, from scratch	none	1e-3 (Adam)
mBERT	General multilingual transformer	bert-base-multilingual-cased	5e-5
XLM-RoBERTa base	General multilingual transformer	xlm-roberta-base	2e-5
AfriBERTa-large	Africa-centric transformer	castorini/afriberta_large	5e-5
AfroXLMR-large	Africa-centric transformer	Davlan/afro-xlmr-large	1e-5

3.6 Training Procedure

All models were trained on the training split with cross-entropy loss and stopped early on the weighted F1 of the deduplicated development set, after which the weights of the best epoch were restored. Every model was trained five times with seeds 42 to 46, so each result is a mean over five runs. Table 3.3 gives the settings. A run that predicts only one class would have been flagged as collapsed; none did.

Table 3.3 Training settings

Setting	Recurrent models	Transformers
Maximum length	64 tokens	128 sub-word tokens
Batch size	64	32 (AfroXLMR-large: 16 × 2 accumulation steps)
Optimiser	Adam	AdamW, weight decay 0.01, linear warm-up over 10% of steps
Maximum epochs	20	10
Early stopping patience	3 epochs	2 epochs
Other	Dropout 0.3, gradient clipping at 1.0	Mixed-precision (fp16) training
Seeds	42, 43, 44, 45, 46	42, 43, 44, 45, 46

3.7 Evaluation Metrics

For each class c, precision P_c, recall R_c and $F1_c$ are:

$$P_c = TP_c / (TP_c + FP_c), \quad R_c = TP_c / (TP_c + FN_c), \quad F1_c = 2\,P_c\,R_c / (P_c + R_c) \tag{3.6}$$

Weighted F1 averages F1_c with weights equal to each class's share of the test set; macro F1 averages them equally; accuracy is the share of tweets classified correctly. Weighted F1 is the main metric because it is the one used by the AfriSenti shared task.

3.8 Statistical Analysis

To compare two models we used a paired bootstrap: test tweets were resampled with replacement 2,000 times, and the difference in weighted F1 between the models' majority-vote predictions (over their five seeds) was recorded each time. A difference is treated as real when its 95% interval excludes zero (Efron and Tibshirani, 1993; Dror et al., 2018). For the effect of code-mixing, the same procedure was applied to the gap between mixed and other tweets, resampling within each group. Because mixed tweets differ from other tweets in label balance and length, we also fitted a logistic regression of whether a prediction is correct on mixing, gold label, the logarithm of tweet length and the presence of Yorùbá diacritics. The prevalence of mixing combines the complete count in S1 with a Wilson interval for the S0 audit.

3.9 Tools

Python 3 with PyTorch and Hugging Face Transformers (Wolf et al., 2020) for the models; scikit-learn for metrics and kappa; statsmodels for the regression; pandas and matplotlib for analysis and figures. Training ran on NVIDIA Tesla T4 GPUs on Google Colab and Kaggle. Appendix D describes the code and how to rerun it.

3.10 Ethical Considerations

No new data were collected. The tweets come from a public benchmark released for research under CC BY 4.0, with user names already masked. Our annotation adds only language labels, keyed by tweet identifier, and contains no tweet text. The two annotators are the authors.

CHAPTER FOUR

RESULTS AND DISCUSSION

4.1 Overall Model Performance

Table 4.1 and Figure 4.1 give the results of all six architectures on the 4,515 test tweets. The two Africa-centric transformers lead, AfroXLMR-large at 73.6 weighted F1 and AfriBERTa-large at 72.5. Their accuracies (73.4 and 72.2) are close to the 74.1 and 72.9 published for these models on Yorùbá in the AfriSenti paper, which suggests our pipeline reproduces the published behaviour. The recurrent models come next, and the two general multilingual transformers come last. The LSTM and AfriBERTa-large were also the most stable across seeds, while XLM-RoBERTa base varied the most.

Table 4.1 Test results of the six models (mean of five seeds; SD = standard deviation of weighted F1)

Model	Weighted F1	SD	Macro F1	Accuracy
AfroXLMR-large	73.6	0.8	71.9	73.4
AfriBERTa-large	72.5	0.5	70.7	72.2
LSTM	67.8	0.4	65.9	67.2
BiLSTM + attention	67.2	1.4	65.3	66.6
mBERT	63.6	1.7	61.8	64.4
XLM-R base	61.7	2.9	59.6	61.8

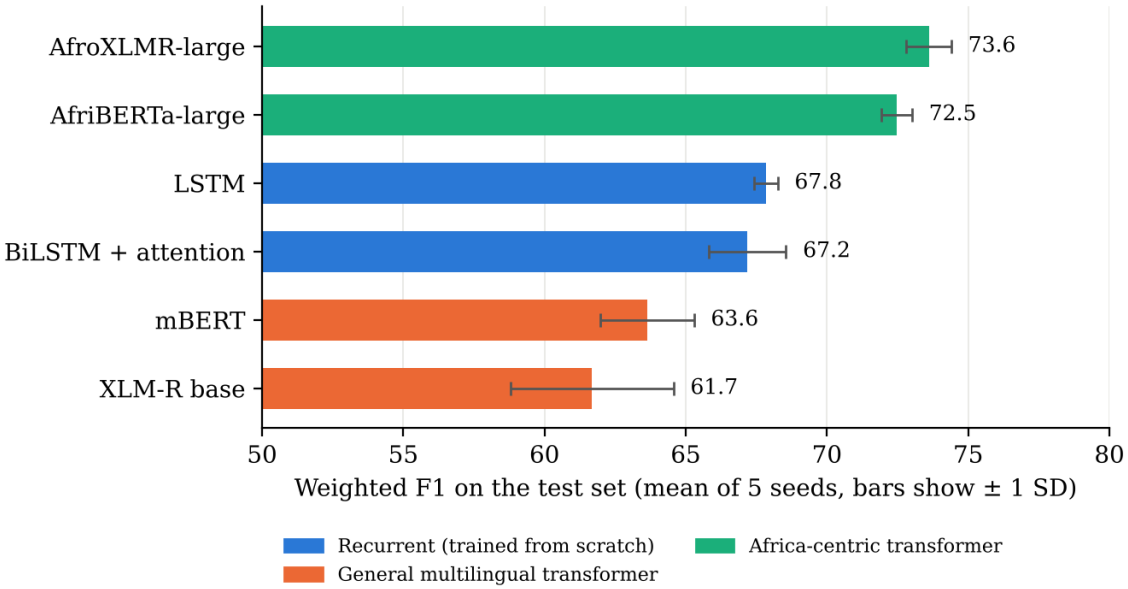


Figure 4.1 Weighted F1 of the six models on the test set

4.2 Performance by Sentiment Class

Figure 4.2 breaks the scores down by class. Positive tweets are the easiest for every model. The neutral class separates the models most: AfroXLMR-large reaches 74.0 F1 on it and the LSTM 69.0, but mBERT and XLM-RoBERTa base fall to 56.5 and 53.3. Neutral tweets carry the fewest sentiment words, so a model needs a good grasp of the language to recognise them, and the general multilingual models, which saw little Yorùbá in pretraining, appear to lack it. The confusion matrices in Figure 4.3 show the same pattern from another angle: the LSTM sends 24% of neutral tweets to the negative class, against 16% for AfroXLMR-large.

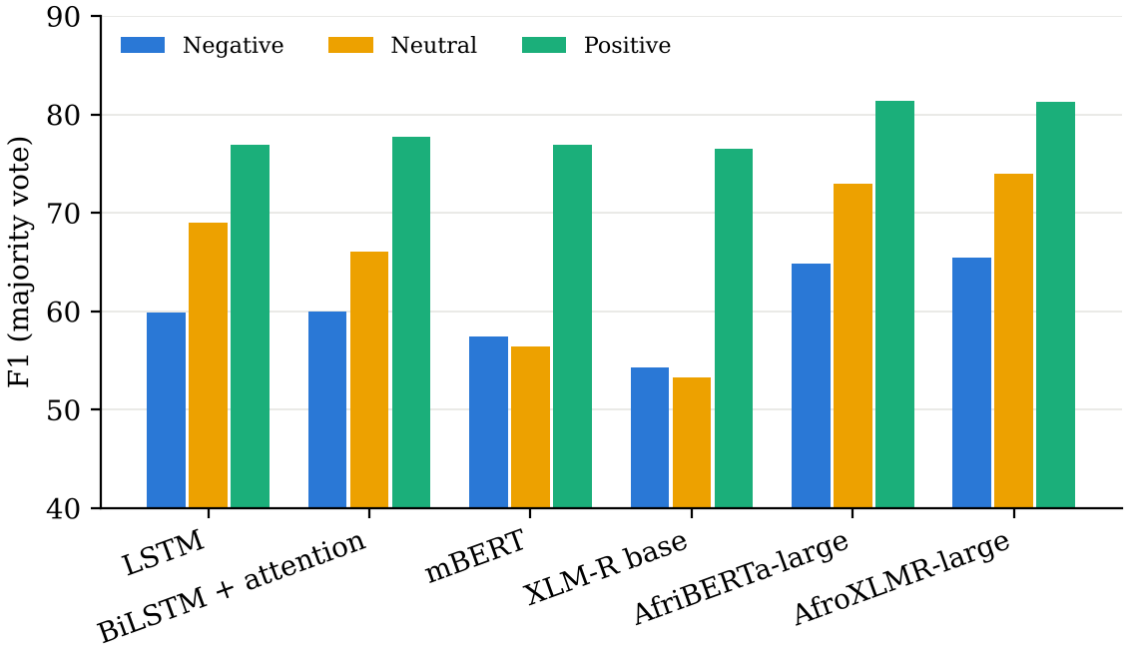


Figure 4.2 F1 by sentiment class (majority vote over five seeds)

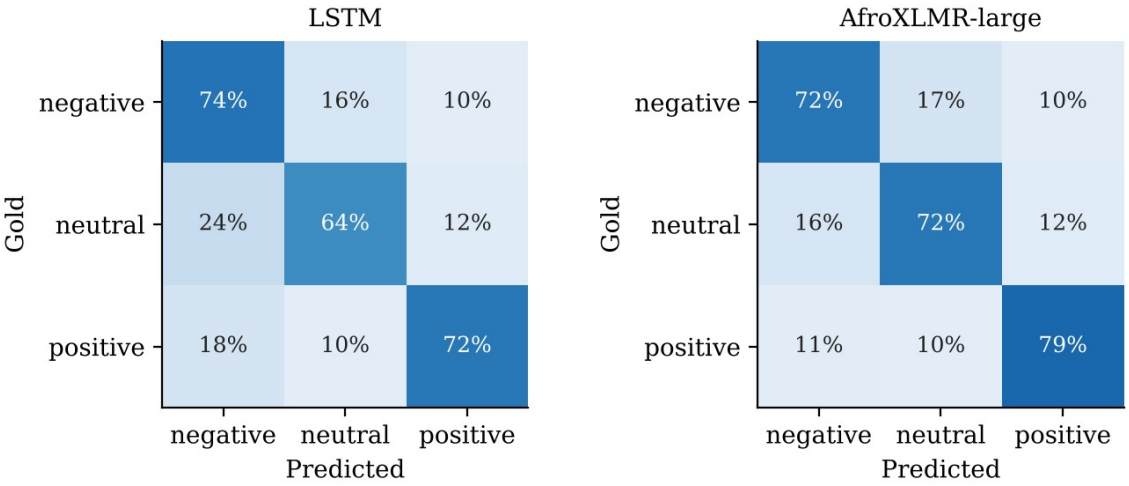


Figure 4.3 Confusion matrices of the LSTM and AfroXLMR-large (rows sum to 100%)

4.3 Statistical Significance of the Differences

Table 4.2 tests the key comparisons. The gap between the two Africa-centric models is within sampling noise, so neither can be called better. AfriBERTa-large beats the LSTM clearly. The LSTM in turn beats both mBERT and XLM-RoBERTa base, and adding bidirectionality and attention did not improve on the plain LSTM.

Table 4.2 Paired bootstrap on majority-vote weighted F1 (difference in points, 95% interval)

Comparison (A minus B)	Difference	95% interval	Significant?
AfroXLMR-large minus AfriBERTa-large	0.5	-0.9 to 1.8	No
AfriBERTa-large minus LSTM	4.4	3.1 to 5.7	Yes
LSTM minus BiLSTM + attention	0.7	-0.3 to 1.7	No
LSTM minus mBERT	5.0	3.5 to 6.5	Yes
LSTM minus XLM-RoBERTa base	7.1	5.6 to 8.5	Yes
mBERT minus XLM-RoBERTa base	2.0	0.6 to 3.4	Yes

4.4 Prevalence of Code-Mixing

The two annotators agreed on 92.8% of the 292 shared tweets, a Cohen's kappa of 0.71 (95% bootstrap interval 0.59 to 0.82), which counts as substantial agreement (Landis and Koch, 1977). They agreed even more closely on which tweets are translation pairs (kappa 0.92). Judging the dominant language of a mixed tweet was harder (kappa 0.50, moderate). The 31 tweets with disagreements or gaps were settled by discussion.

Table 4.3 and Figure 4.4 show what the 1,978 flagged tweets turned out to be. Only 269 (13.6%) were genuinely mixed, and 312 (15.8%) were translation pairs; the rest were judged not mixed. None of the 400 audited zero-score tweets was mixed. Mixed tweets therefore make up 6.0% of the test set (95% interval 6.0% to 6.5%), and translation pairs another 6.9%. More than half of the mixed tweets are led by Yorùbá (144 mostly Yorùbá, 70 balanced, 55 mostly English). The lexicon index missed no mixed tweet in our audit, but only 14% of the tweets it flags are mixed, so on its own it would overstate mixing about seven times.

Table 4.3 Annotation outcome

Group	Tweets	Annotated	Mixed	Translation pair	Not mixed
S1 (index above zero)	1,978	1,978	269	312	1,397
S0 (index of zero)	2,537	400 (audit)	0	0	400

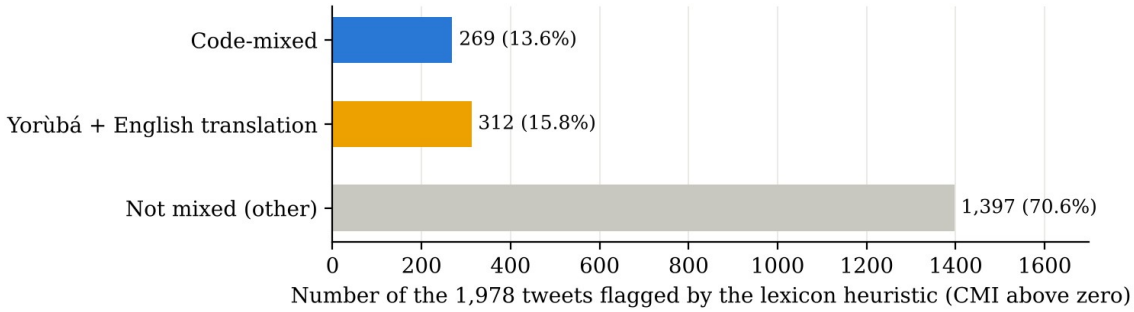


Figure 4.4 What the tweets flagged by the Code-Mixing Index turned out to be

4.5 Effect of Code-Mixing on Sentiment Accuracy (RQ1)

Table 4.4 and Figure 4.5 compare each model's weighted F1 on the 269 mixed tweets with its score on every other test tweet. Five of the six models score lower on mixed tweets. The drop is about four to five points for the LSTM, the BiLSTM with attention and mBERT, and about one point or less for the Africa-centric models; XLM-RoBERTa base is the exception and scores higher on mixed tweets. None of these gaps is statistically significant: every 95% interval includes zero, because 269 tweets is too few to detect gaps of this size with confidence.

Table 4.4 Weighted F1 on mixed and other tweets (mean of five seeds; interval from the bootstrap)

Model	Mixed tweets	Other tweets	Gap	95% interval
AfroXLMR-large	73.2	73.6	-0.4	-5.5 to 5.3
AfriBERTa-large	71.3	72.5	-1.3	-5.5 to 5.0
LSTM	63.6	68.1	-4.5	-9.7 to 1.9
BiLSTM + attention	62.5	67.5	-4.9	-9.1 to 2.6
mBERT	59.2	63.9	-4.7	-10.5 to 1.7
XLM-R base	64.8	61.5	+3.3	-3.2 to 9.1

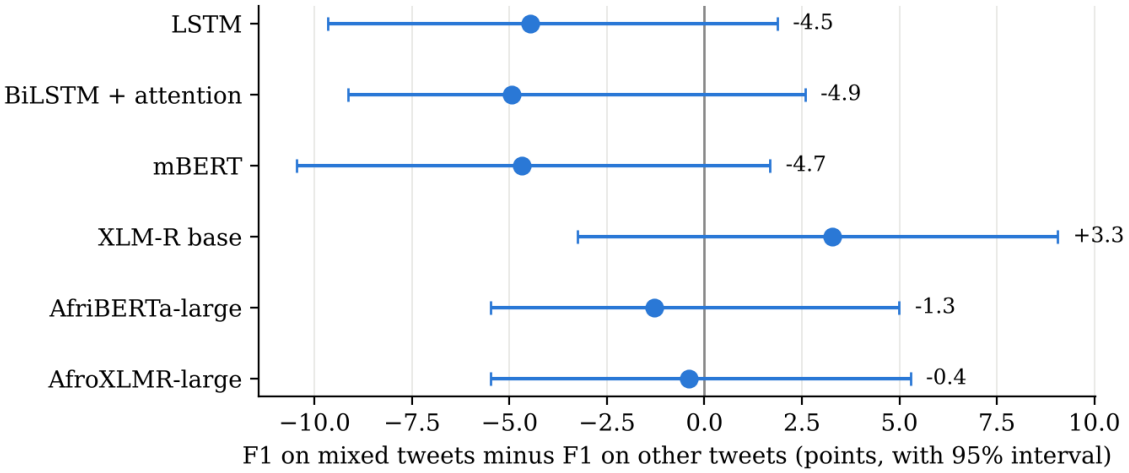


Figure 4.5 Gap between mixed and other tweets for each model, with 95% intervals

The logistic regression confirms this. After holding the gold label, tweet length and the use of diacritics fixed, the odds of a correct prediction on a mixed tweet range from 0.77 to 1.16 times those on other tweets, and none differs from 1 at the 5% level. Mixed tweets are also more often positive than other tweets (49% against 43%), which is why this control matters.

The answer to RQ1 is therefore that, on this benchmark, code-mixing does not measurably change how well the models interpret sentiment. The direction of the gaps is still informative: models that learnt only from the training data, or that know little Yorùbá, lose more on mixed tweets than the Africa-centric models do. XLM-RoBERTa base, the weakest model overall, does better on mixed tweets than on the rest, probably because English carries more of the meaning in them; on mostly-English mixed tweets its F1 rises to 77.9, against 59.3 on mostly-Yorùbá ones.

4.6 Local Expressions and Idioms (RQ3)

To test whether contextual embeddings help with local expressions, we took the 86 test tweets that explicitly involve a proverb, idiom, saying or praise poem (for example, tweets containing òwe, proverb, oríkì, idiom or ìtumọ̀). Table 4.5 and Figure 4.6 show that these tweets are hard for every model. The two recurrent models, which have no contextual embeddings, lose 15 and 13 points on them. AfroXLMR-large, the strongest contextual model, loses only 6 points and reaches 67.6, the best score on these tweets. Its advantage over the LSTM grows from about 5.6 points on other tweets to about 14.6 points on local expressions.

Table 4.5 Weighted F1 on tweets with proverbs, idioms or praise poetry (n = 86) and on other tweets

Model	Local expressions	Other tweets	Drop
AfroXLMR-large	67.6	73.7	6.1
AfriBERTa-large	62.9	72.7	9.8
LSTM	53.0	68.1	15.2
BiLSTM + attention	54.5	67.4	12.9
mBERT	58.4	63.7	5.4
XLM-R base	56.3	61.8	5.5

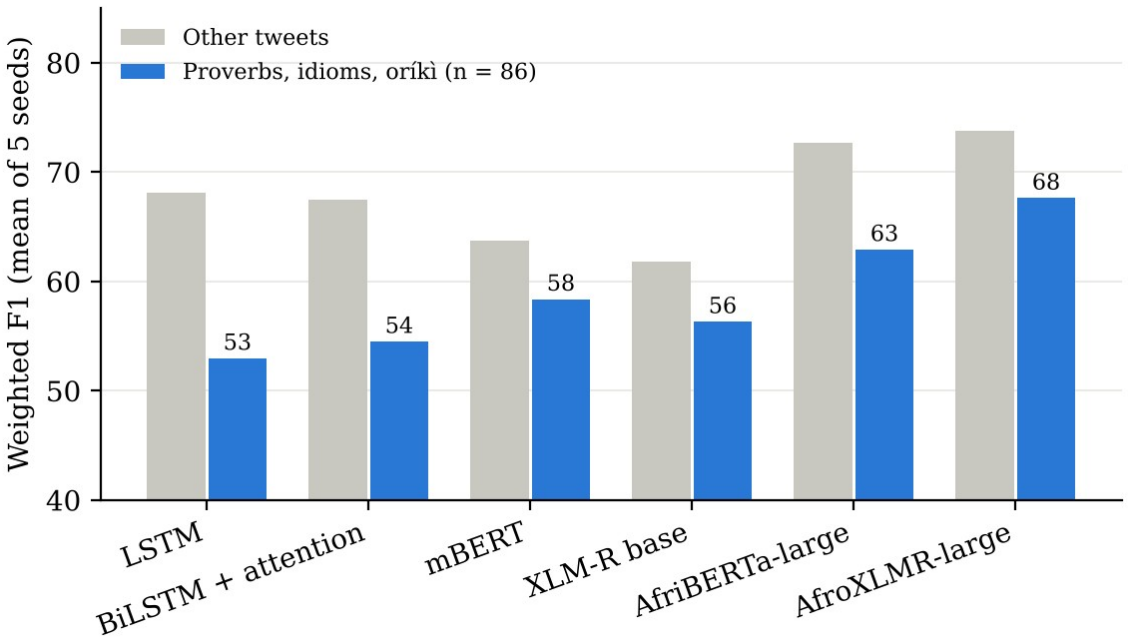


Figure 4.6 Weighted F1 on local-expression tweets compared with other tweets

With only 86 such tweets the extra advantage has a wide interval (-2.6 to 20.7 points) and is not significant on its own. The pattern is nonetheless consistent across models: every transformer loses less on these tweets than either recurrent model. The answer to RQ3 is a qualified yes: contextual embeddings, especially from a model adapted to African languages, cope noticeably better with proverbs and idioms than embeddings learnt from scratch, although these expressions remain the weakest point for all the models.

4.7 Error Analysis of the Best Model

AfroXLMR-large, by majority vote, misclassifies 67 of the 269 mixed tweets. We examined every one and grouped them by their most likely cause (Table 4.6). 50 of the 67 errors involve the neutral class, so the hard boundary is between neutral and polar sentiment rather than between positive and negative.

Table 4.6 Causes of AfroXLMR-large errors on mixed tweets

Cause	Errors
Informational or educational post	25
Proverb, idiom or oral literature	17
Slang, pidgin, banter or insult	12
Other	10
Questionable gold label	2
Sentiment differs across the two languages	1

The largest group is informational or educational posts such as word-of-the-day glosses, programme announcements and news. The gold labels and the model disagree about whether such posts carry sentiment at all; a post promoting the Yorùbá language may be labelled positive while the model reads it as neutral information. The second group is proverbs, idioms and oríkì, whose sentiment depends on cultural knowledge. One tweet says that the snail cannot run as fast as the dog, which the annotators read as encouragement (slow progress still arrives) and the model as a neutral statement. The third group is slang, Nigerian Pidgin and banter, where teasing and insults are easily misread. Only one error came from the two languages pulling in opposite directions. None of the main causes is specific to switching between languages, which agrees with the finding of Section 4.5.

4.8 What the Attention Model Looks At

Table 4.7 lists the three words with the highest attention weights for a few test tweets (seed 42). In clear cases the attention falls on sensible words: for the complaint about fuel and transport prices it picks owó (money), ọkọ̀ (vehicle) and epo (fuel), and for a prayer for peace it picks alaafia (peace). The failures are just as telling. In "ẹ̀rù kankan ò bàmí" (no fear frightens me), the model attends to ẹ̀rù (fear) and predicts negative, missing that the sentence negates the fear and is in fact confident and positive. Negation and idiom are exactly what a word-level model without contextual pretraining finds hardest, which matches the results of Section 4.6.

Table 4.7 Most-attended words of the BiLSTM with attention (seed 42)

Tweet (abridged)	Gold	Predicted	Top attention words
owó epo léwó owó ọkọ̀ léwó	negative	negative	owó, ọkọ̀, epo
mo fe alaafia fun venezuela jòwó jòwó...	positive	positive	venezuela, alaafia, fun
àti pé ẹ̀rù kankan ò bàmí ilé baba kíí bọmọ lẹ́rù	positive	negative	kankan, ẹ̀rù, ò
ai fagba fẹni kan ni koje kí ayé gun	negative	positive	gun, kí, ayé
ẹ jọ̀wọ́ kí ni pọ̀mọ́ ṣe o	neutral	positive	jọ̀wọ́, ẹ, kí

4.9 Discussion

Three findings stand out. First, pretraining on the right languages matters more than model size or architecture alone. AfriBERTa-large, a much smaller model than AfroXLMR-large, matches it statistically and beats the general multilingual models by 9 to 11 points, and even a plain LSTM trained only on the 8,522 training tweets beats mBERT and XLM-RoBERTa base. This agrees with the AfriSenti results and with the argument behind AfriBERTa that targeted data can outweigh scale (Ogueji et al., 2021).

Second, code-mixing is a smaller problem than expected, both in frequency and in effect. About one test tweet in sixteen is genuinely mixed, and the best models handle those tweets about as well as the rest. Much of what a lexicon heuristic calls mixing is in fact Yorùbá text with an English translation attached, a common pattern in educational and cultural posts.

Third, the errors that remain are about culture and pragmatics: proverbs, praise poetry, slang and posts whose sentiment is debatable. For a company or government agency using these models, this means the main risk is not that a model will misread a tweet because it switches to English, but that it will misread a proverb or a sarcastic remark. We also observed that development-set scores overstate the weaker models: mBERT and XLM-RoBERTa base scored about 71 on the development set but 62 to 64 on the test set, so choosing a model by development score alone would be risky.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

This project studied deep neural networks for sentiment analysis of code-mixed English-Yorùbá tweets. We audited the AfriSenti Yorùbá benchmark, built a human code-mixing annotation of its test set with substantial agreement between annotators, and trained six models, two recurrent networks and four transformers, five times each under one protocol. We compared the models overall, by class, on mixed and non-mixed tweets and on tweets built around proverbs and idioms, tested every key difference statistically, and analysed the errors of the best model and the attention of the BiLSTM.

5.2 Conclusion

RQ1: code-mixing occurs in about 6% of the test tweets, and it does not measurably lower the accuracy of any of the six models; the Africa-centric models lose about one point on mixed tweets and the others about four to five, but none of these gaps is significant on this test set.

RQ2: the Africa-centric transformers, AfroXLMR-large and AfriBERTa-large, perform best and are statistically tied. A plain LSTM outperforms the general multilingual transformers, and adding bidirectionality and attention to the LSTM does not help.

RQ3: contextual embeddings help with local expressions. On tweets built around proverbs, idioms and praise poetry, the recurrent models lose 13 to 15 points while AfroXLMR-large loses about 6, although the sample is too small for the difference to be significant on its own and these expressions remain the hardest case for every model.

5.3 Recommendations

- For brand monitoring, public opinion analysis and election or policy sentiment tracking in Yorùbá, use an Africa-centric transformer such as AfriBERTa-large, which matches the best model while being smaller and faster to run.
- Do not rely on general multilingual models such as mBERT or XLM-RoBERTa base for Yorùbá without testing them on local data; a simple LSTM did better here.
- Do not use a lexicon-based mixing index to estimate how much code-mixing a corpus contains; a small human audit changes the answer several times over.

- Pay particular attention to proverbs, sarcasm and slang when checking a model's output, and consider adding more such examples to training data.
- Report results over several random seeds and test differences statistically, because single runs can mislead.

5.4 Contributions to Knowledge

- A human code-mixing annotation of the AfriSenti Yorùbá test set (2,378 tweets), with agreement figures, released for reuse.
- The first measurement, to our knowledge, of how common English-Yorùbá mixing is in this benchmark and of its effect on six architectures.
- Evidence that a lexicon heuristic overstates mixing about sevenfold and confuses translation with mixing.
- A controlled, multi-seed comparison showing that Africa-centric pretraining outweighs model size, and a data audit showing that 12.9% of development tweets duplicate training tweets.

5.5 Limitations

The data come from one platform. Code-mixing was labelled per tweet, not per word, and only on the test set. The test set contains only 269 mixed tweets and 86 explicit local-expression tweets, which limits how small an effect we can detect. The local-expression subset was found by keywords, so it captures tweets that talk about proverbs or idioms rather than every tweet that uses one. Each model used one training recipe without extensive tuning. The annotators are also the authors.

5.6 Suggestions for Further Work

Future work could label code-mixing word by word to find which switch points cause errors, annotate a larger set of mixed tweets from the training data or other platforms, build a proper evaluation set of Yorùbá proverbs and idioms with sentiment labels, and extend the analysis to Hausa, Igbo and Nigerian Pidgin, which AfriSenti also covers.

REFERENCES

Alabi, J. O., Adelani, D. I., Mosbach, M., and Klakow, D. (2022). Adapting pre-trained language models to African languages via multilingual adaptive fine-tuning. In Proceedings of the 29th International Conference on Computational Linguistics (COLING 2022). https://aclanthology.org/2022.coling-1.382

Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In Proceedings of the International Conference on Learning Representations (ICLR 2015).

Cohen, J. (1960). A coefficient of agreement for nominal scales. Educational and Psychological Measurement, 20(1), 37-46.

Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., and Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. In Proceedings of ACL 2020.

Das, A., and Gambäck, B. (2014). Identifying languages at the word level in code-mixed Indian social media text. In Proceedings of the 11th International Conference on Natural Language Processing (ICON 2014).

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of NAACL-HLT 2019.

Dodge, J., Ilharco, G., Schwartz, R., Farhadi, A., Hajishirzi, H., and Smith, N. (2020). Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. arXiv:2002.06305.

Dror, R., Baumer, G., Shlomov, S., and Reichart, R. (2018). The hitchhiker's guide to testing statistical significance in natural language processing. In Proceedings of ACL 2018.

Efron, B., and Tibshirani, R. J. (1993). An Introduction to the Bootstrap. Chapman and Hall.

Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780.

Joshi, P., Santy, S., Budhiraja, A., Bali, K., and Choudhury, M. (2020). The state and fate of linguistic diversity and inclusion in the NLP world. In Proceedings of ACL 2020.

Landis, J. R., and Koch, G. G. (1977). The measurement of observer agreement for categorical data. Biometrics, 33(1), 159-174.

Liu, B. (2012). Sentiment Analysis and Opinion Mining. Morgan and Claypool.

Muhammad, S. H., Adelani, D. I., Ruder, S., Ahmad, I. S., Abdulmumin, I., Bello, B. S., Choudhury, M., Emezue, C. C., Abdullahi, S. S., Aremu, A., Jorge, A., and Brazdil, P. (2022). NaijaSenti: A Nigerian Twitter sentiment corpus for multilingual sentiment analysis. In Proceedings of LREC 2022. https://aclanthology.org/2022.lrec-1.63/

Muhammad, S. H., Abdulmumin, I., Ayele, A. A., et al. (2023b). AfriSenti: A Twitter sentiment analysis benchmark for African languages. In Proceedings of EMNLP 2023. https://aclanthology.org/2023.emnlp-main.862/

Muhammad, S. H., Abdulmumin, I., Yimam, S. M., et al. (2023c). SemEval-2023 Task 12: Sentiment analysis for African languages (AfriSenti-SemEval). In Proceedings of SemEval-2023. arXiv:2304.06845.

Ogueji, K., Zhu, Y., and Lin, J. (2021). Small data? No problem! Exploring the viability of pretrained multilingual language models for low-resourced languages. In Proceedings of the 1st Workshop on Multilingual Representation Learning (MRL 2021).

Pang, B., and Lee, L. (2008). Opinion mining and sentiment analysis. Foundations and Trends in Information Retrieval, 2(1-2), 1-135.

Patwa, P., Aguilar, G., Kar, S., Pandey, S., PYKL, S., Gambäck, B., Chakraborty, T., Solorio, T., and Das, A. (2020). SemEval-2020 Task 9: Overview of sentiment analysis of code-mixed tweets. In Proceedings of SemEval-2020.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In Advances in Neural Information Processing Systems 30 (NeurIPS 2017).

Winata, G. I., Aji, A. F., Yong, Z.-X., and Solorio, T. (2023). The decades progress on code-switching research in NLP: A systematic survey on trends and challenges. In Findings of ACL 2023.

Wolf, T., Debut, L., Sanh, V., et al. (2020). Transformers: State-of-the-art natural language processing. In Proceedings of EMNLP 2020: System Demonstrations.

APPENDIX A: TRAINING ENVIRONMENT AND RUN TIMES

All runs used an NVIDIA Tesla T4 GPU (16 GB) on Google Colab or Kaggle, PyTorch 2.11 and Hugging Face Transformers 5.16 to 5.18. Every run records its seed, library versions, GPU, training time and best epoch in results/runs/ of the code package.

Table A.1 Average training time and best epoch per run

Model	Minutes per run	Mean best epoch
AfroXLMR-large	26.7	6.4
AfriBERTa-large	6.7	4.0
LSTM	0.4	11.6
BiLSTM + attention	0.4	5.4
mBERT	12.0	5.6
XLM-R base	28.1	6.8

APPENDIX B: RESULTS OF EVERY RUN

Weighted F1 (percent) on the full test set for each of the 30 runs.

Table B.1 Test weighted F1 by seed

Model	Seed 42	Seed 43	Seed 44	Seed 45	Seed 46
AfroXLMR-large	74.4	74.3	72.6	73.1	73.7
AfriBERTa-large	72.4	72.2	71.9	73.4	72.5
LSTM	68.6	67.6	67.7	67.7	67.6
BiLSTM + attention	67.7	65.0	68.7	67.0	67.5
mBERT	62.3	65.3	63.7	65.2	61.6
XLM-R base	62.6	62.3	59.3	58.5	65.7

APPENDIX C: CODE-MIXING ANNOTATION GUIDELINES

Unit: one tweet. Do not look at the sentiment of the tweet; judge only the languages used.

is_codemixed = 1 when the writer naturally mixes Yorùbá and English, within a sentence or across sentences, including an English sentence quoted inside Yorùbá text.

is_codemixed = 0 when the tweet is in one language only; when English appears only in hashtags, links, names or user names; when the only English is one ordinary borrowed word such as "phone", "okay" or "daddy"; or when a Yorùbá line is followed by its English translation (write TRANS in the notes column).

If English is added that is not a translation of the Yorùbá, such as a comment, a reaction or a title like "today's Yorùbá motivation", the tweet is mixed.

dominance (only for mixed tweets): YO when Yorùbá makes up about two thirds of the words or more, EN when English does, BAL otherwise. Hashtags, links and names are not counted.

Tweets marked as shared are labelled by both annotators without discussion; disagreements are settled together afterwards.

APPENDIX D: CODE PACKAGE AND HOW TO RUN IT

The accompanying code package contains everything needed to reproduce this report:

- notebooks/01_data_cmi_annotation_baseline.ipynb: loads the data, audits duplicates, computes the Code-Mixing Index and builds the annotation sheets.
- notebooks/02_train_all_models.ipynb: trains the six models with five seeds each and saves every prediction (needs a GPU).
- notebooks/03_results_and_figures.ipynb: produces every table and figure in Chapter Four from the saved predictions (no GPU needed).
- src/analysis.py: agreement, prevalence, the bootstrap tests and the logistic regression.
- data/annotations/: the code-mixing labels, the double annotation before discussion, the error categories and the guidelines.
- results/: scores, predictions and run records of all 30 runs, plus the figures.

To reproduce Chapter Four on any computer: install the packages in requirements.txt, run python data/download_afrisenti.py, then open notebooks/03_results_and_figures.ipynb and run all cells. The core of the recurrent models is shown below.

```
class RNNClassifier(nn.Module):
    def __init__(self, vocab, bidirectional, attention,
                 emb=200, hid=128, n_classes=3, dropout=0.3):
        super().__init__()
        self.attention = attention
        self.bidirectional = bidirectional
        self.emb = nn.Embedding(vocab, emb, padding_idx=0)
        self.rnn = nn.LSTM(emb, hid, batch_first=True,
                           bidirectional=bidirectional)
        d = hid * (2 if bidirectional else 1)
        if attention:
            self.W = nn.Linear(d, d)
            self.v = nn.Linear(d, 1, bias=False)
        self.drop = nn.Dropout(dropout)
        self.fc = nn.Linear(d, n_classes)

    def forward(self, x, lens):
        e = self.drop(self.emb(x))
        packed = pack_padded_sequence(e, lens.cpu(), batch_first=True,
                                      enforce_sorted=False)
        out, (h, _) = self.rnn(packed)
        out, _ = pad_packed_sequence(out, batch_first=True,
                                     total_length=x.size(1))
        w = None
        if self.attention:                        # equation (3.4)
            s = self.v(torch.tanh(self.W(out))).squeeze(-1)
            s = s.masked_fill(x == 0, -1e9)       # ignore padding
            w = torch.softmax(s, dim=1)
            rep = (w.unsqueeze(-1) * out).sum(1)
        elif self.bidirectional:
            rep = torch.cat([h[-2], h[-1]], 1)
        else:
            rep = h[-1]
        return self.fc(self.drop(rep)), w
```